SMART WASTE MANAGEMENT SYSTEM (SWMS-MAS)

DALI Multi-Agent System

Author: Lokwo Denis
Student ID: 295895
Course: Argent Architecture
Institution: University of L'Aquila

Date: March 5, 2026

Contents

1 Introduction **5**

2 Problem Context and Motivation **5**

3 System Objectives, Scope, and Constraints **6**
- 3.1 Primary Objectives . . . 6
- 3.2 Scope of this Project Phase . . . 6
- 3.3 Explicit Constraints . . . 6

4 Methodological Framing (GAIA-Oriented Analysis) **6**

5 Full Architecture Description **6**
- 5.1 Layered Architecture . . . 6
- 5.2 Architecture Diagram . . . 7
- 5.3 Key Operational Characteristics . . . 8

6 Agent Organization, Roles, and Behavioral Model **8**
- 6.1 Virtual Organization . . . 8
- 6.2 Role Table . . . 8
- 6.3 Behavioral Signatures . . . 8

7 UML Sequence Diagram and Detailed Flow Interpretation **9**
- 7.1 Sequence Diagram . . . 9
- 7.2 Flow Interpretation . . . 10

8 UML Collaboration Diagram and Communication Semantics **10**

9 Event Table (External/Internal + Source) **10**

10 Action Table and Operational Logic **12**

11 Message Flow and End-to-End Lifecycle Scenarios **13**
- 11.1 Nominal (Successful) Scenario . . . 13
- 11.2 Refusal Scenario . . . 13
- 11.3 Timeout Scenario . . . 14

12 Reliability, Fault Handling, and Recovery Strategy **14**
- 12.1 Reliability Mechanisms . . . 14
- 12.2 Operational Health Strategy . . . 14

13 Installation **14**
- 13.1 Prerequisites . . . 14
- 13.2 MAS Runtime Commands . . . 14
- 13.3 Web MVP Commands . . . 15
- 13.4 Expected Runtime Topology . . . 15

14 Execution Evidence (Screenshots to Include) **16**

15 Web Integration Layer (MVP) **17**
15.1 Implemented MVP Capabilities 17
15.2 Why Non-Invasive Matters 18

16 Results, Critical Discussion, and Limitations **18**
16.1 Achieved Results 18
16.2 Design Strengths 18
16.3 Current Limitations 18
16.4 Improvement Opportunities 18

17 Conclusion **18**

List of Figures

1 Architecture Diagram 7
2 Sequence Diagram 9
3 Smart Bin Class Abstraction Model 10
4 TMUX session overview 16
5 Web Dashboard Control System Section 16
6 Collection report 17
7 Graphical representation section 17

List of Tables

1 Role-to-Agent Responsibility Mapping 8
2 Smart Bin Agent Event Specification 10
3 Control Center Agent Event Specification 11
4 Truck Agent Event Specification 11
5 Logger Agent Event Specification 11
6 Control Center Agent Action Specification 12
7 Smart Bins Agent Action Specification 12
8 Trucks Agent Action Specification 13
9 Logger Agent Action Specification 13

1 Introduction

The **SMART WASTE MANAGEMENT SYSTEM (SWMS-MAS)** is a DALI-based Multi-Agent System implemented on SICStus Prolog to model autonomous municipal waste collection. The project formalizes waste logistics as an event-driven cooperative process among heterogeneous agents with distinct responsibilities: smart bins (sensing and alert generation), trucks (resource execution and decision response), a control center (global coordination and policy enforcement) and a logger (observability and traceability).

The system demonstrates the full service loop from environment-state sensing to operational closure:

- bins autonomously increase fill-level and emit status telemetry,
- full bins trigger service requests,
- The control center dispatches trucks through a robust negotiation pattern
- trucks accept or refuse according to local state and probabilistic policy,
- accepted tasks proceed through working/completion phases,
- control center clears serviced bins and closes the cycle,
- The logger records all key events for monitoring and post-run analysis.

From an engineering perspective, SWMS-MAS includes resilience mechanisms (retry queues, cooldown windows, inflight watchdog, timeout fallback), operational scripts (start/stop/restart/healthcheck), and a non-invasive web wrapper MVP for control and live visibility. This makes the project suitable as a prototype for practical smart-city orchestration workflows.

Project Repository

The complete implementation of the Smart Waste Management System (SWMS), including agent logic, coordination mechanisms, and experimental setup, is available on GitHub.

Repository Link: https://github.com/Lokwo12/SWMS-MAS

2 Problem Context and Motivation

Urban waste collection is rarely a static scheduling problem; it is a dynamic coordination problem with asynchronous demand, uncertain resource availability, and potential communication/response delays. Conventional centralized planning alone is often brittle under real-time perturbations. Multi-Agent Systems provide a natural formalism for this domain because they allow:

- local autonomy: each operational entity can reason/act independently,
- distributed interaction: entities negotiate via explicit protocols,
- event-driven adaptation: state changes propagate through message flow,
- fault tolerance by design: retries, alternative assignments, fallback policies.

The present project adopts this paradigm by representing core actors as software agents and designing explicit interaction protocols around fill alerts, dispatch negotiation, service execution, and closure. The resulting system is analyzable, modular, and extensible.

3 System Objectives, Scope, and Constraints

3.1 Primary Objectives

1. Implement autonomous bin monitor and full detection
2. coordinate trucks assignment through a formal agent interaction model.
3. Handle negative outcomes(refuse/timeout), without deadlock.
4. guarantee cycle closure through clear and reset logic
5. provide transparent event level observability and health checks;
6. Support operator level control via scripts and web wrapper.

3.2 Scope of this Project Phase

- Execution of 8 agents + Linda server runtime.
- script life cycle management via shell commands.
- tmux-base process topology and visual monitoring.
- Event parsing and streaming for web dashboard MVP.

3.3 Explicit Constraints

- No physical IoT hardware integration in current phase.
- Collection policy is simulation-driven (no road network optimization).
- System behavior is deterministic/probabilistic via configured rules, not ML-driven.

4 Methodological Framing (GAIA-Oriented Analysis)

The system design follows GAIA-compatible abstractions:

- Role model: Coordinator, Collector, Sensor/Producer, Observer.
- Interaction model:message protocols (‘inform(...)‘) among role instances.
- Services model:role responsibilities mapped to internal predicates/actions.
- Organizational rules:one central dispatcher, multiple resource executors, event logger.

This framing ensures end-to-end traceability from the conceptual analysis phase to concrete implementation artifacts. For instance, the *GAIA* role responsibility assigned to the *Coordinator* is operationalized through the predicates `process_full_events/1`, `dispatch_request/1`, and `tick_inflight_watchdog/1` within the control-center agent type.

5 Full Architecture Description

5.1 Layered Architecture

The complete system can be interpreted in five layers:

1. Infrastructure Layer: OS process management, tmux sessions, SICStus runtime, shell scripts.
2. MAS Core Layer: DALI agents and Linda communication substrate.

3. Coordination Layer: Runtime dispatch logic, retries, timeout watchdog, bin/truck lifecycle management.
4. Observability Layer: Logger event stream, healthcheck verification, operational diagnostics.
5. Web Wrapper Layer (MVP): API orchestration, WebSocket event feed, dashboard visualization.

5.2 Architecture Diagram

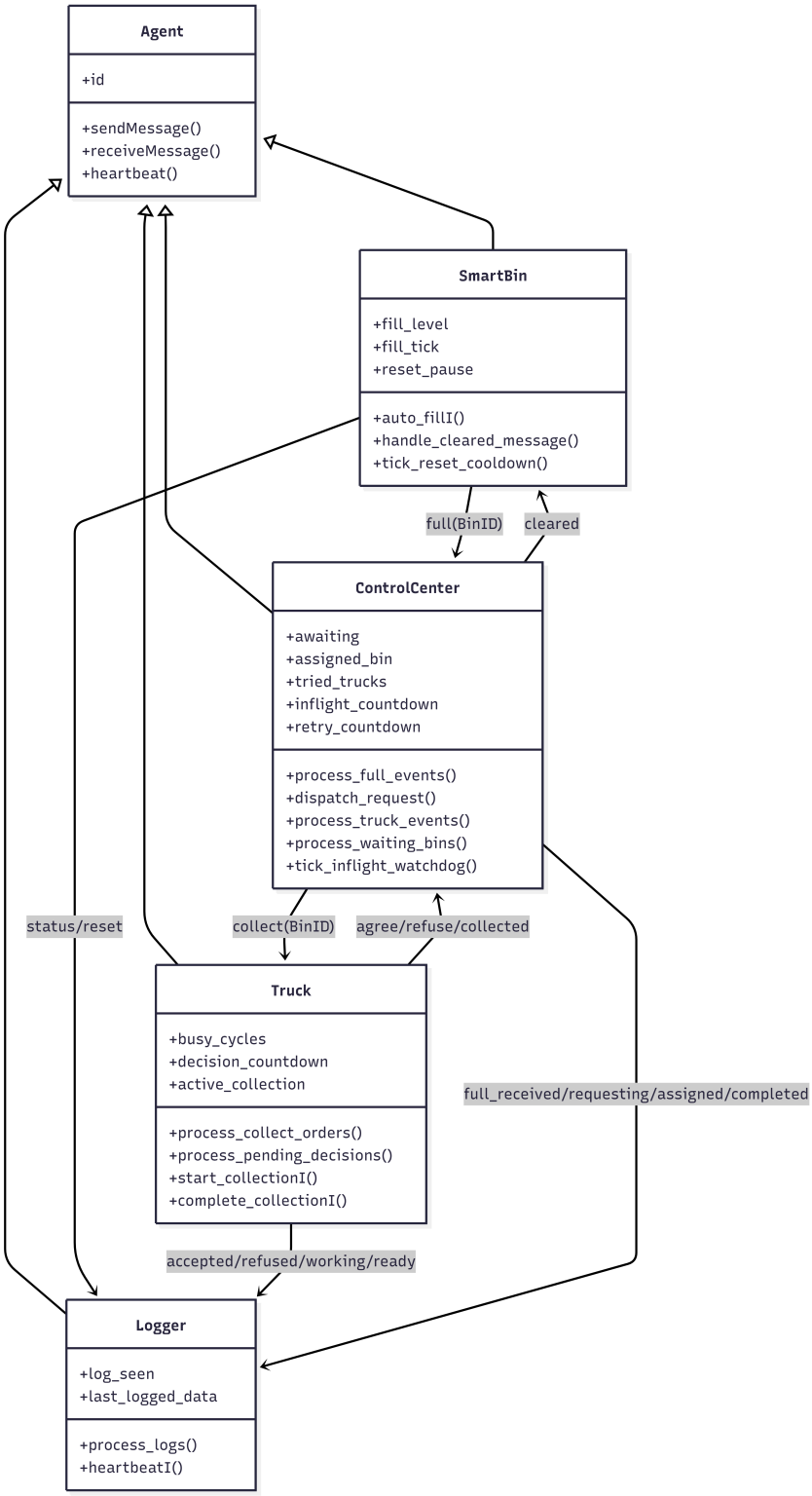

Figure 1: Architecture Diagram

5.3 Key Operational Characteristics

- Startup script automatically prepares environment and launches expected topology.
- Healthcheck validates both process-level and signal-level liveness.
- Runtime behavior relies on asynchronous event consumption from ‘past(...)‘ memory.
- Control-center state machines prevent dispatch races and stale inflight assignments

6 Agent Organization, Roles, and Behavioral Model

6.1 Virtual Organization

The organization is **centralized-coordination with distributed execution**

- one coordinator (‘control_center‘) governs assignment policy,
- three producers (‘smart_bin1..3‘) generate service demand,
- three collectors (‘truck1..3‘) provide execution capacity,
- one observer (‘logger‘) records event semantics.

6.2 Role Table

Table 1: Role-to-Agent Responsibility Mapping

Role	**Agent(s)**	**Responsibilities**
Coordinator	`control_center`	Full-event intake, dispatch decision, retry handling, timeout watchdog, and request closure.
Collector	`truck1`, `truck2`, `truck3`	Accept or refuse collection orders, perform service operations, and send completion notifications.
Sensor	`smart_bin1`, `smart_bin2`, `smart_bin3`	Fill-level progression, status emission, full alert triggering, and reset upon clearance.
Observer	`logger`	Lifecycle event logging, deduplication, and liveness heartbeat monitoring.

6.3 Behavioral Signatures

- Smart bins: periodic internal progression with emission of status and full.
- Trucks: stateful availability + probabilistic decision under order input.
- Control center:queue-driven, timer-governed, retry-capable dispatcher.
- Logger: passive consumer with selective noise suppression

7 UML Sequence Diagram and Detailed Flow Interpretation

7.1 Sequence Diagram

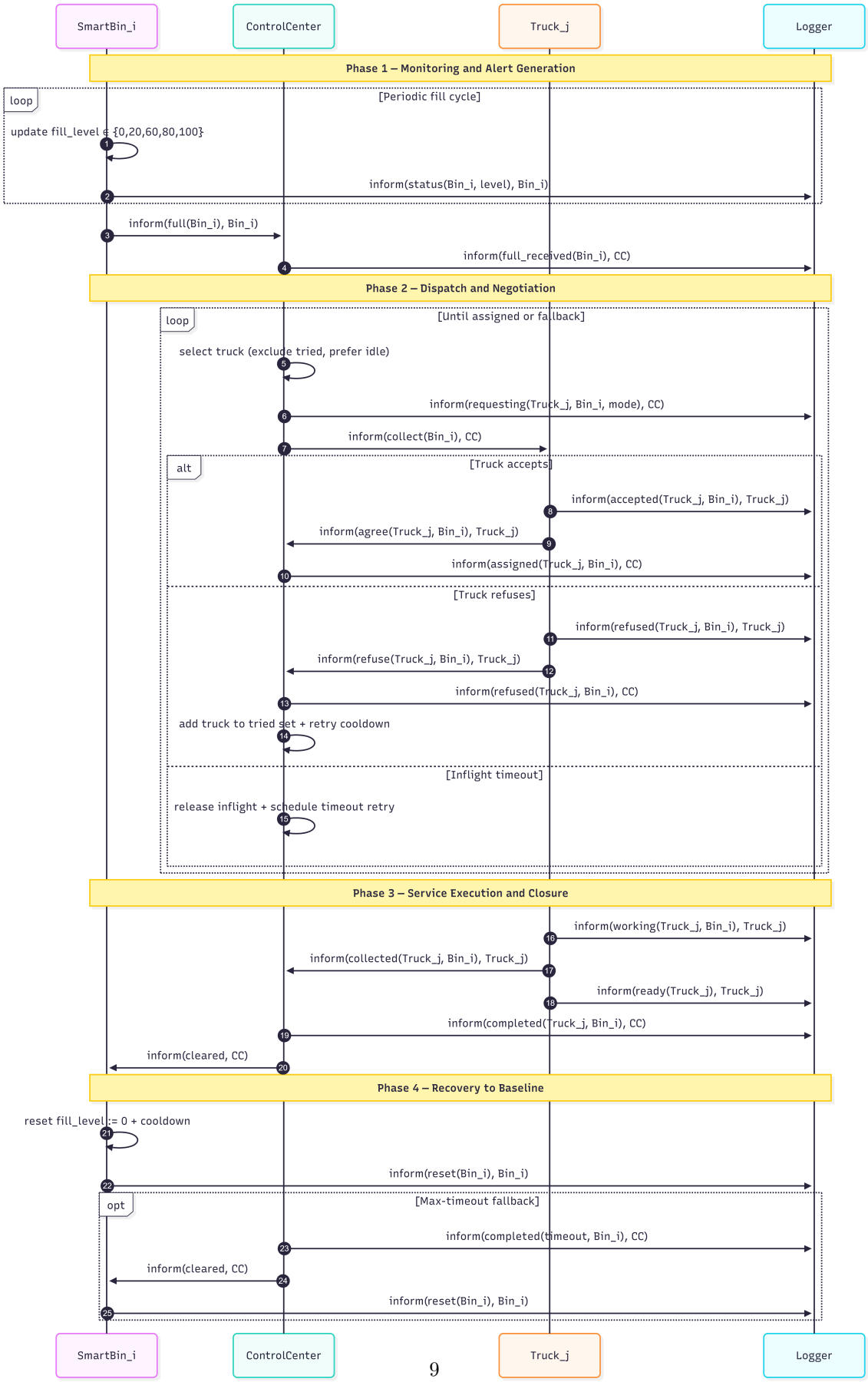

Figure 2: Sequence Diagram

7.2 Flow Interpretation

- **Phase 1:**demand emergence through environment sensing.
- **Phase 2:**constrained assignment under uncertain resource willingness
- **Phase 3:** successful execution and closure notifications.
- **Phase 3:**successful execution and closure notifications.
- **Phase 4:** restoration of initial local state for next cycle.
- **Fallback branch:** ensures liveness by preventing indefinite waiting.

8 UML Collaboration Diagram and Communication Semantics

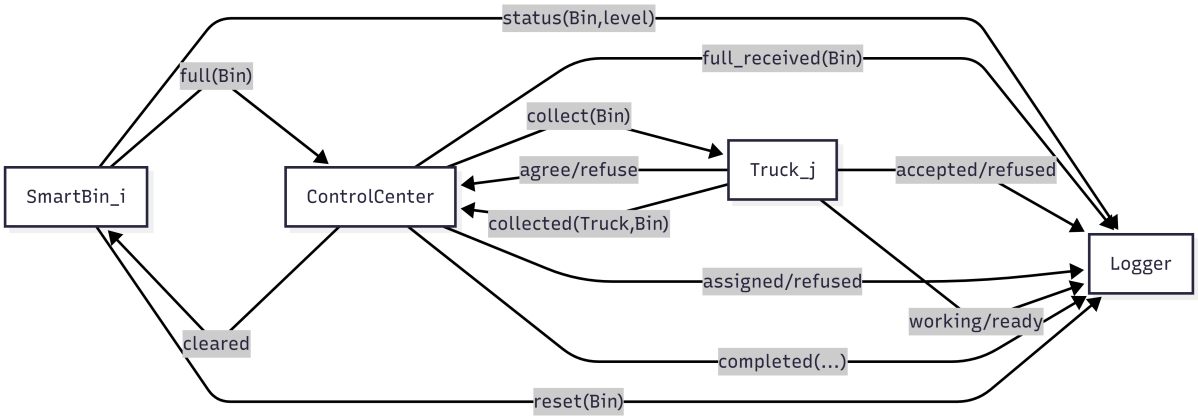

Figure 3: Smart Bin Class Abstraction Model

Class Model Notes.
This class diagram represents an abstraction layer over the underlying logic predicates. It exposes stateful properties (e.g., fill level, cooldown state) and functional services (e.g., fill progression, reset handling), enabling conceptual mapping from predicate-level implementation to architectural design notation.

9 Event Table (External/Internal + Source)

Table 2: Smart Bin Agent Event Specification

Agent	Event	Type	Source / Operational Logic
`smart_bin`	`status(BinID, Level)`	External (outgoing message)	`auto_fill/1` periodic fill cycle
`smart_bin`	`full(BinID)`	External (outgoing message)	`auto_fill/1` when level reaches 100%
`smart_bin`	`cleared(BinID)`	External (incoming message)	`inform_EE/2` received from control center
`smart_bin`	`reset(BinID)`	External (outgoing message)	`handle_cleared_message/1` after clear processing
`smart_bin`	`fill_step_allowed`	Internal (timer trigger)	`allow_fill_step/0` under `fill_tick` cadence
`smart_bin`	`reset_cooldown_active`	Internal (timer trigger)	`tick_reset_cooldown/1` managing reset pause

Table 3: Control Center Agent Event Specification

Agent	Event	Type	Source / Operational Logic
control_center	full(BinID)	External (incoming message)	process_full_events/1 from bin alerts
control_center	agree(TruckID, BinID)	External (incoming message)	process_truck_events/1 upon truck acceptance
control_center	refuse(TruckID, BinID)	External (incoming message)	process_truck_events/1 upon truck refusal
control_center	collected(TruckID, BinID)	External (incoming message)	process_truck_events/1 upon completion callback
control_center	collect(BinID)	External (outgoing message)	dispatch_request/1 via cc_send_collect/2
control_center	cleared(BinID)	External (outgoing message)	cc_send_cleared/1 after completion or timeout fallback
control_center	dispatch_countdown_elapsed	Internal (timer trigger)	tick_dispatch_queue/1 and process_waiting_bins/1
control_center	retry_countdown_elapsed	Internal (timer trigger)	tick_retry_queue/1 and process_waiting_bins/1
control_center	inflight_timeout	Internal (watchdog trigger)	tick_inflight_watchdog/1 handling dispatch expiration

Table 4: Truck Agent Event Specification

Agent	Event	Type	Source / Operational Logic
trucks	collect(BinID)	External (incoming message)	process_collect_orders/1 from control center
trucks	agree(TruckID, BinID)	External (outgoing message)	process_pending_decisions/1 upon acceptance
trucks	refuse(TruckID, BinID)	External (outgoing message)	process_pending_decisions/1 upon rejection
trucks	collected(TruckID, BinID)	External (outgoing message)	complete_collection/1 after service completion
trucks	decision_countdown_elapsed	Internal (timer trigger)	tick_decision_queue/1 and process_pending_decisions/1
trucks	collection_start_elapsed	Internal (timer trigger)	tick_collection_start/1 and start_collection/1

Table 5: Logger Agent Event Specification

Agent	Event	Type	Source / Operational Logic
logger	alive(LoggerID)	External (outgoing message)	heartbeatI/1 periodic heartbeat channel
logger	inform(Data, From)	External (incoming message)	process_logs/1 over past(...) history
logger	log_seen(T)	Internal (state trigger)	process_logs/1 seen-event deduplication

- External events model inter-agent communication boundary crossings.

- Internal events model predicate-level runtime transitions that affect future behavior without immediate external messaging.

10 Action Table and Operational Logic

Table 6: Control Center Agent Action Specification

Agent	Action	Description
`control_center`	`process_full_events/1`	Consumes full-bin alerts, marks the bin as awaiting service, initializes dispatch state, and logs `full_received`.
`control_center`	`dispatch_request/1`	Selects a candidate truck (busy/tried filtering), marks the request as inflight, and sends `collect(BinID)`.
`control_center`	`process_truck_events/1`	Handles `agree`, `refuse`, and `collected` outcomes, updating assignment, retry, and completion state.
`control_center`	`tick_inflight_watchdog/1`	Monitors inflight timeout; schedules retry or enforces timeout fallback completion after threshold.
`control_center`	`process_waiting_bins/1`	Dispatches queued bins when dispatch or retry countdowns expire.

Table 7: Smart Bins Agent Action Specification

Agent	Action	Description
`smartbins`	`auto_fillI/1`	Advances the fill level, emits `status(BinID, Level)`, and triggers `full(BinID)` when the threshold reaches 100%.
`smartbins`	`handle_cleared_message/1`	Resets the fill level and activates cooldown state upon reception of `cleared(BinID)`.
`smartbins`	`tick_reset_cooldown/1`	Advances the reset cooldown timer before normal filling activity resumes.

Table 8: Trucks Agent Action Specification

Agent	Action	Description
trucks	process_collect_orders/1	Reads incoming collect(BinID) requests and enqueues a decision countdown for evaluation.
trucks	process_pending_decisions/1	Applies the acceptance policy and emits agree or refuse, including telemetry notifications to the logger.
trucks	start_collectionI/1	Emits working(TruckID, BinID) when the collection-start delay reaches zero.
trucks	complete_collectionI/1	Emits collected(TruckID, BinID) to the control center and ready(TruckID) to the logger when service completes.
trucks	tick_busy/1	Decrements busy cycles and governs truck availability state transitions.

Table 9: Logger Agent Action Specification

Agent	Action	Description
logger	process_logs/1	Consumes incoming inform(Data, From) messages, filters noise, performs deduplication, and prints structured lifecycle flow traces.
logger	heartbeatI/1	Sends a periodic liveness heartbeat to the control_center.

Action-Level Interpretation Each action can be interpreted as a service realization of role responsibilities. Coordination quality emerges from the composition of actions, not from any single predicate.

11 Message Flow and End-to-End Lifecycle Scenarios

11.1 Nominal (Successful) Scenario

1. Bin reaches full threshold and alerts control center.
2. Control center dispatches a truck and logs request.
3. Truck accepts and starts work.
4. Truck reports collection completion.
5. Control center logs completion and sends clear.
6. Bin resets and resumes monitoring.

11.2 Refusal Scenario

1. Dispatched truck refuses due to local state/policy.
2. Control center records refusal, updates tried set.
3. Retry timer triggers another dispatch attempt.
4. Alternate truck is selected and process continues.

11.3 Timeout Scenario

1. Inflight order receives no valid progress in watchdog window.
2. Control center clears inflight state and schedules retry.
3. Repeated timeouts increment timeout counter.
4. Threshold reached leads to fallback closure (completed(timeout, BinID)).

12 Reliability, Fault Handling, and Recovery Strategy

12.1 Reliability Mechanisms

1. **Retry cooldown queues** prevent aggressive re-dispatch loops.
2. **Busy filtering**reduces assignment conflict probability.
3. **Inflight watchdog** detects stalled service progression.
4. **Timeout fallback** prevents indefinite queue lock.

12.2 Operational Health Strategy

healthcheck.sh validates:

- SICStus process liveness,
- tmux session presence,
- expected pane topology,
- lifecycle signal appearance in
- logger output.

This dual structural + behavioral health approach is stronger than process-only checks.

13 Installation

13.1 Prerequisites

- tmux installed and available in PATH.
- SICStus Prolog 4.6.x installed.
- Project cloned with executable script permissions.
- Linux, WSL2, or macOS shell environment. Default path expected in startup script:

Environment. Default path expected in startup script:
/usr/local/sicstus4.6.0/bin/sicstus
If required, edit SICSTUS_HOME in startmas.sh.

13.2 MAS Runtime Commands

Here are the MAS management commands:

```
chmod +x startmas.sh stopmas.sh restartmas.sh healthcheck.sh
./startmas.sh
./healthcheck.sh
./stopmas.sh
```

13.3 Web MVP Commands

One-Command Mode Scripts

Execute the following scripts for one-command operations:

```
./run_all.sh
./stop_all.sh
./deep_restart.sh
```

Manual Mode

Backend Setup

```
cd web_mvp/backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8080 --reload
```

Frontend Setup

```
cd web_mvp/frontend
python3 -m http.server 5173
```

Dashboard Access

Open the following URL in your browser:

```
http://localhost:5173
```

13.4 Expected Runtime Topology

TMUX Session Setup

Session: *DALI_session*

Pane Titles:

- *LINDA_SERVER*
- *control_center*
- *logger*
- *smart_bin1*, *smart_bin2*, *smart_bin3*
- *truck1*, *truck2*, *truck3*

14 Execution Evidence (Screenshots to Include)

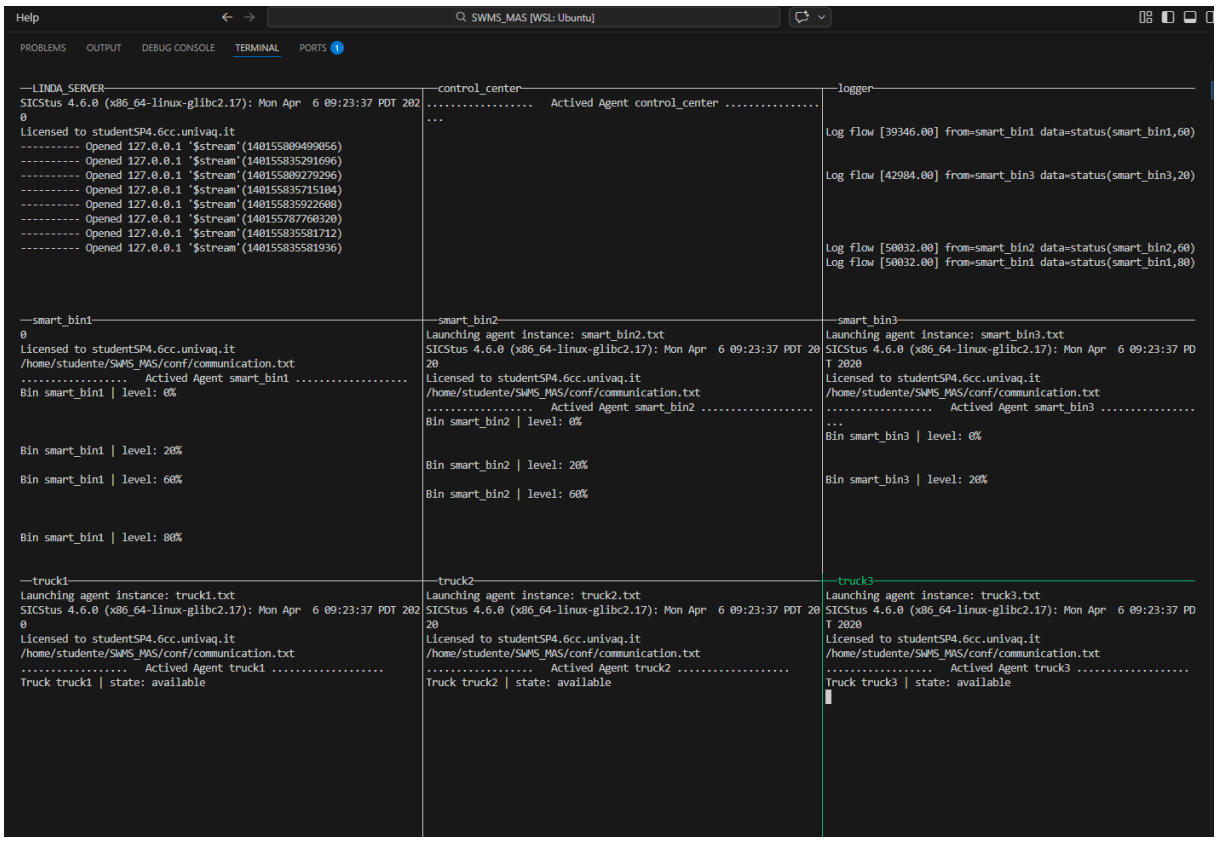

Figure 4: TMUX session overview

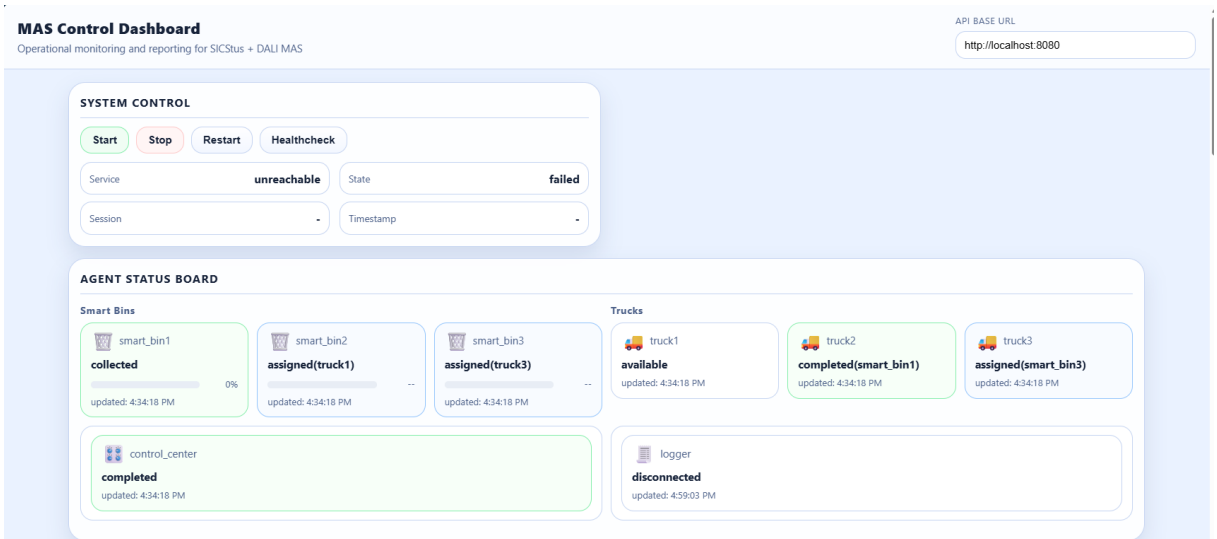

Figure 5: Web Dashboard Control System Section

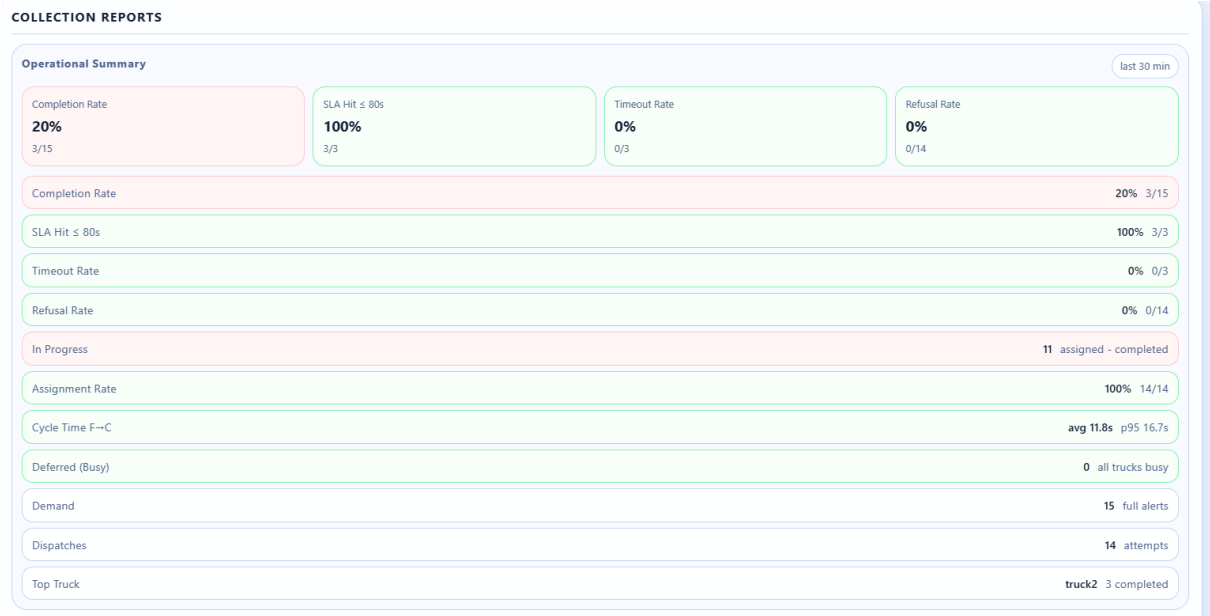

Figure 6: Collection report

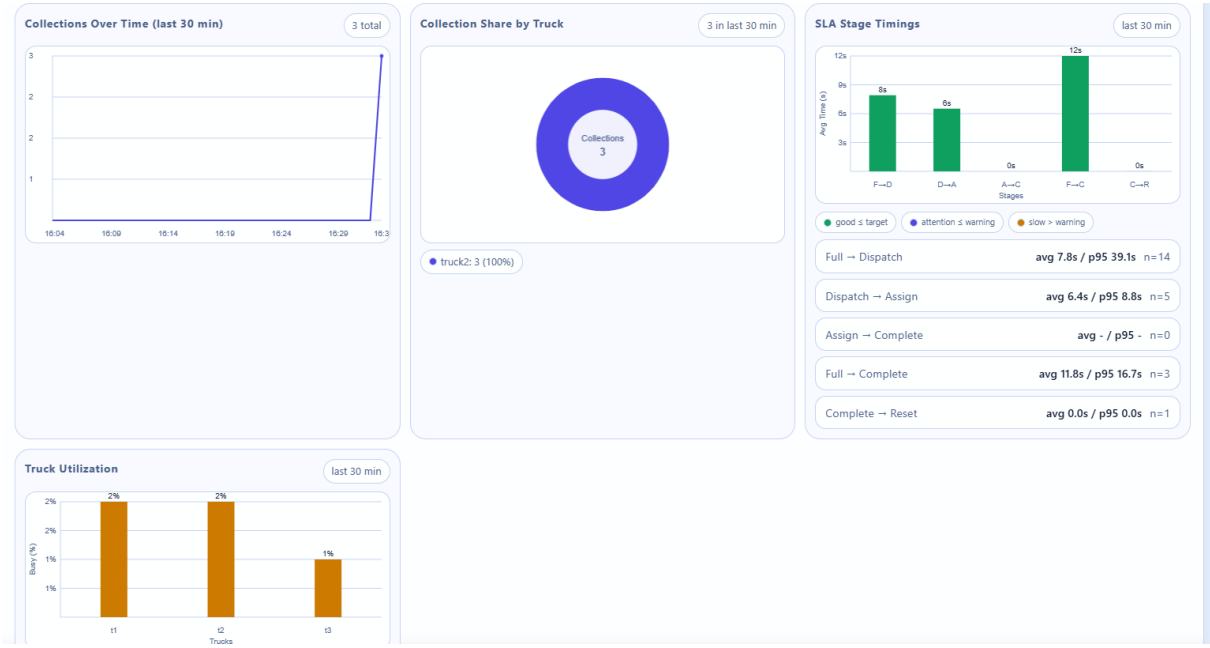

Figure 7: Graphical representation section

15 Web Integration Layer (MVP)

The web layer is intentionally non-invasive and wraps existing MAS scripts and logs.

15.1 Implemented MVP Capabilities

- REST endpoints for start/stop/restart/healthcheck.
- WebSocket stream of normalized lifecycle events.
- Dashboard with control panel and live event console.

15.2 Why Non-Invasive Matters

- preserves proven MAS logic,
- reduces regression risk,
- accelerates operational visibility,
- enables phased migration toward the production architecture.

16 Results, Critical Discussion, and Limitations

16.1 Achieved Results

- Integrated End-to-end autonomous waste-collection cycle.
- Robust handling of refusals and delayed responses.
- Formal mapping between role theory and code-level behavior.
- Operational controls are available via the shell and web interfaces.

16.2 Design Strengths

- clear role separation, transparent event semantics,
- practical resilience controls,
- maintainable type/instance organization.

16.3 Current Limitations

- no GIS-aware route optimization, no direct integration with
- physical sensor telemetry,
- no persistent long-term analytics store in core MVP.

16.4 Improvement Opportunities

- adaptive dispatch policy based on truck history,
- predictive fullness modeling,
- event-sourcing persistence for KPI dashboards,
- scenario simulation harness for quantitative benchmarking.

17 Conclusion

SWMS_MAS demonstrates that urban waste collection can be effectively modeled as a resilient multi-agent coordination problem. The project combines theoretical MAS foundations with practical implementation detail: role-driven behavior, protocol-based interactions, runtime watchdogs, and operational observability.

References

[1] COSTANTINI, Stefania. *The DALI Agent-Oriented Logic Programming Language: Summary and References*. 2015.

[2] COSTANTINI, S.; TOCCHIO, A. *A Logic Programming Language for Multi-Agent Systems*. Logics in Artificial Intelligence.

[3] De Gasperis, G., Costantini, S., Nazzicone, G. *DALI Multi-Agent Systems Framework*. GitHub Repository. Available at: https://github.com/AAAI-DISIM-UnivAQ/DALI